

Trabalho Prático: Sudoku

Neste trabalho, vamos construir um Sudoku inteligente. Em vez de escrevermos código super complexo para o computador adivinhar os números, vamos usar uma ferramenta da Google (o OR-Tools).

A nossa única tarefa é explicar ao computador as regras do jogo. A regra de ouro do Sudoku é muito simples: **num grupo de células, os números não se podem repetir**. A ferramenta da Google percebe esta regra perfeitamente se a modelarmos como um problema de lógica.

Em algumas partes foi usado o auxílio de uma LLM, pode se verificar como forma de comentario as partes em questao. Aqui esta o link para a conversa com a LLM : <https://share.gemini.google/3hXBsoW7E5to>

1. O Saco Genérico (Regra R1)

A classe `Box` é como se fosse um saco vazio. Nós podemos atirar células lá para dentro.

Este saco não quer saber se as células vão formar uma linha, uma coluna ou um quadrado. Ele só serve para duas coisas:

1. Guardar a posição (linha e coluna) e o número que lá está.
2. Dar um estalo (erro) se tentarmos meter lá dentro uma coordenada ou um número que não existe no tabuleiro de Sudoku.

reusable 

2. Linhas, Colunas e Quadrados (Regras R2 e R3)

Como já temos o nosso saco genérico (`Box`), não precisamos de inventar a roda. Vamos criar dois sacos especiais que sabem preencher-se sozinhos:

- **O Quadrado (Cube):** Se lhe dissermos qual é o bloco que queremos (ex: o bloco do canto superior esquerdo), ele faz as contas e mete as 9 células desse quadrado lá para dentro.
- **A Reta (Path):** Nós dizemos onde começa e onde acaba, e ele vai a andar em linha reta (na horizontal ou na vertical) a apanhar as células todas pelo caminho. É assim que fazemos as linhas e as colunas do jogo!

3. Sortear as Pistas (Regra R4)

Ninguém joga Sudoku com um tabuleiro totalmente vazio. Precisamos de alguns números para começar.

Esta função espalha alguns números (pistas) à sorte pelo tabuleiro. Para garantir que não tenta colocar dois números no mesmo buraco, usamos uma função do Python que sorteia posições sem as repetir. O resultado final é devolvido num saco simples (Box).



4. O Cérebro e a Fábrica (Regras R5 e R6)

A função `solve_sudoku_csp` é o nosso **cérebro** (R5). Ela recebe os sacos todos cheios de células e diz ao computador: *"Olha, em cada um destes sacos, os números não se podem repetir. As pistas têm de ficar onde estão. Agora, preenche o resto e descobre a solução!"*.

A função `build_and_solve_sudoku` é a nossa **fábrica** (R6). Ela cria as 9 linhas, as 9 colunas e os 9 quadrados, atira as pistas lá para o meio, e manda esse molho todo de regras para o cérebro resolver.

5. Prova dos 9 (Testes)

Como sortearmos os números iniciais completamente às cegas, o gerador pode ter o azar de meter um '5' na mesma linha que outro '5', tornando o jogo impossível logo de início.

Para que os testes funcionem sem encravar, pedimos ao computador para tentar inventar jogos repetidamente até encontrar um que tenha lógica. Quando encontrar, passamos o jogo resolvido a pente fino: vemos se alguma regra falhou ou se as pistas foram roubadas do sítio.

A provar que tudo funciona:

- ✓ Teste 1 (Limites): Passou! Rejeitou coordenadas que não existem.
- ✓ Teste 2 (Resolver Sudoku): Passou! Verificámos as linhas e colunas todas e a matemática está perfeita.
- ✓ Teste 3 (Escalar n=2): Passou! O código funciona perfeitamente para grelhas 4x4.

O Tabuleiro Final:

```
[8, 9, 6, 5, 3, 1, 4, 7, 2]
[7, 2, 5, 6, 8, 4, 1, 9, 3]
[1, 4, 3, 9, 2, 7, 6, 8, 5]
[6, 7, 9, 1, 4, 5, 2, 3, 8]
[3, 8, 1, 2, 7, 9, 5, 4, 6]
[2, 5, 4, 3, 6, 8, 7, 1, 9]
[9, 1, 2, 4, 5, 3, 8, 6, 7]
[5, 3, 7, 8, 1, 6, 9, 2, 4]
[4, 6, 8, 7, 9, 2, 3, 5, 1]
```

